TIMSORT

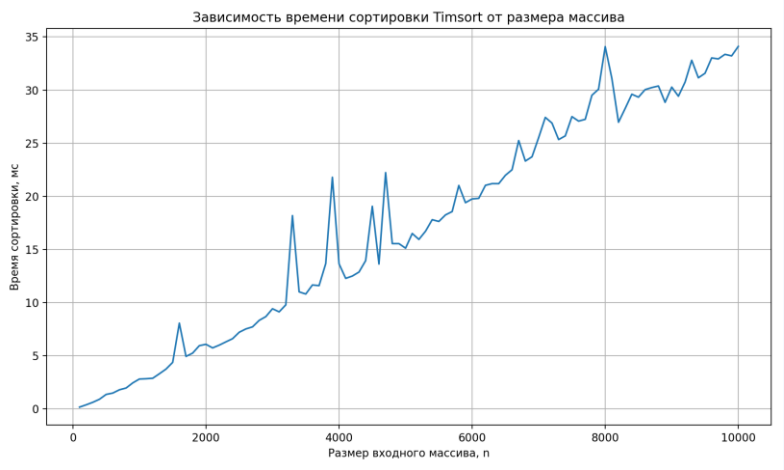
Семестровая работа № 2

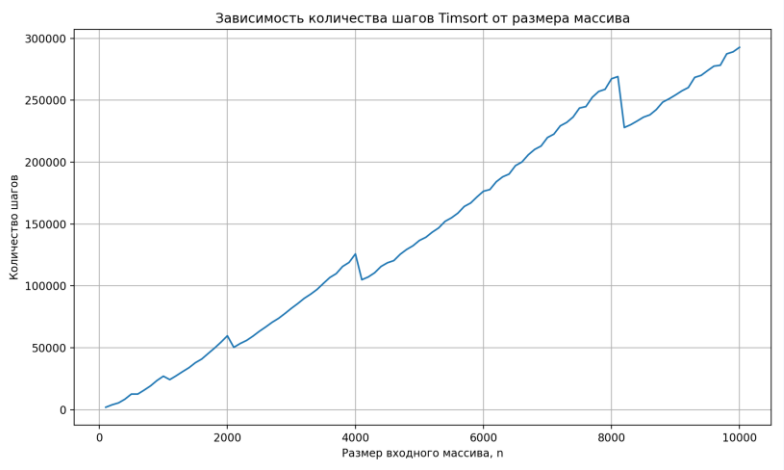
Анализ и реализация алгоритма сортировки на Python

В работе показаны идея TimSort, измерение времени и числа шагов, а также анализ результатов.

Алгоритмы и структуры данных

Два основных результата эксперимента





Цель работы и что нужно было сделать

- реализовать выбранный алгоритм сортировки TimSort
- сгенерировать 100 наборов входных данных и сохранить их в отдельные файлы;
- считать данные из файла перед запуском сортировки;
- измерить только работу сортировки: по времени и по количеству шагов;
- построить графики и сделать выводы по сложности алгоритма.

Что такое TimSort

Ключевая идея алгоритма

- TimSort — это гибридная сортировка, которая сочетает идеи сортировки вставками и слиянием.
- Алгоритм ищет в массиве уже готовые упорядоченные участки, серии (runs).
- Короткие серии удобно досортировывать вставками, а затем последовательно сливать.
- За счёт этого TimSort особенно хорошо работает на частично упорядоченных данных.

Как устроена моя программа

Три основных файла проекта

data_generator.py
создаёт 100 файлов со случайными числами

→

data/input
каждый набор данных сохраняется в .txt

→

benchmark.py
читает файлы, меряет время и шаги

→

timsort.py
выполняет саму сортировку

Ключевые части реализации

- find_run() ищет естественную серию и разворачивает убывающий участок;
- insertion_sort() досортировывает короткие серии до min_run;
- merge() сливает соседние уже отсортированные части;
- timsort() управляет поиском серий и поэтапным слиянием.

```
def timsort(arr, count_steps=False):
    n = len(arr)
    counter = [0]
    min_run = get_min_run(n)
    runs = []

    i = 0
    while i < n:
        run_len = find_run(arr, i, n, counter)
        if run_len < min_run:
            right = min(i + min_run - 1, n - 1)
            insertion_sort(arr, i, right, counter)
            run_len = right - i + 1
        runs.append((i, run_len))
        i += run_len
```

Как проводился эксперимент

Методика измерений

01

Генерация

100 файлов со случайными целыми числами

02

Чтение

каждый файл считывается в список Python

03

Измерение

время только для timsort(), без вспомогательных операций

04

Сохранение

таблица результатов и два графика

100 … 10 000
диапазон размеров n

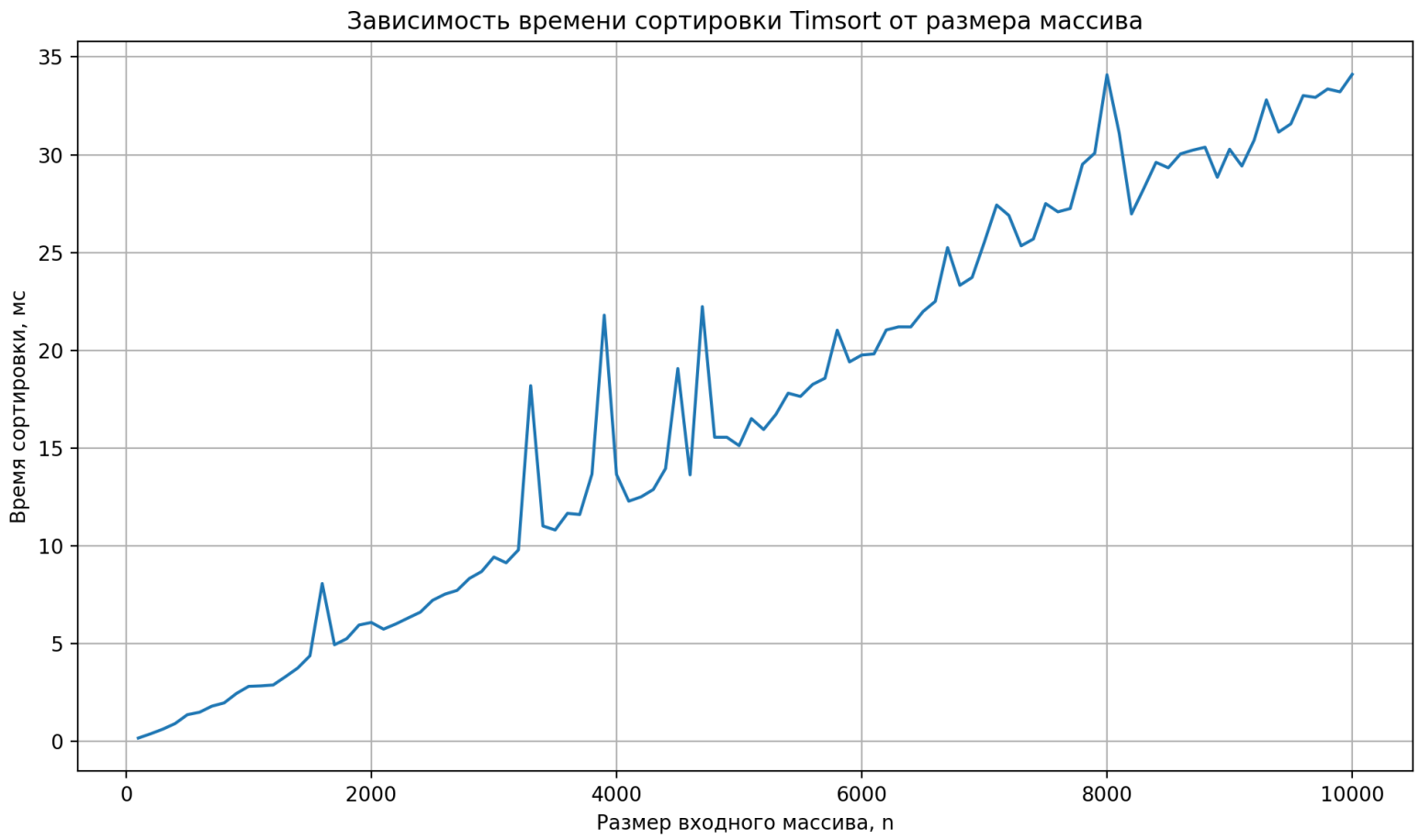
3 запуска
для оценки времени

медиана
устраняет случайные пики

counter[0]
считает шаги внутри алгоритма

Результат 1: время сортировки

Зависимость времени выполнения от размера массива



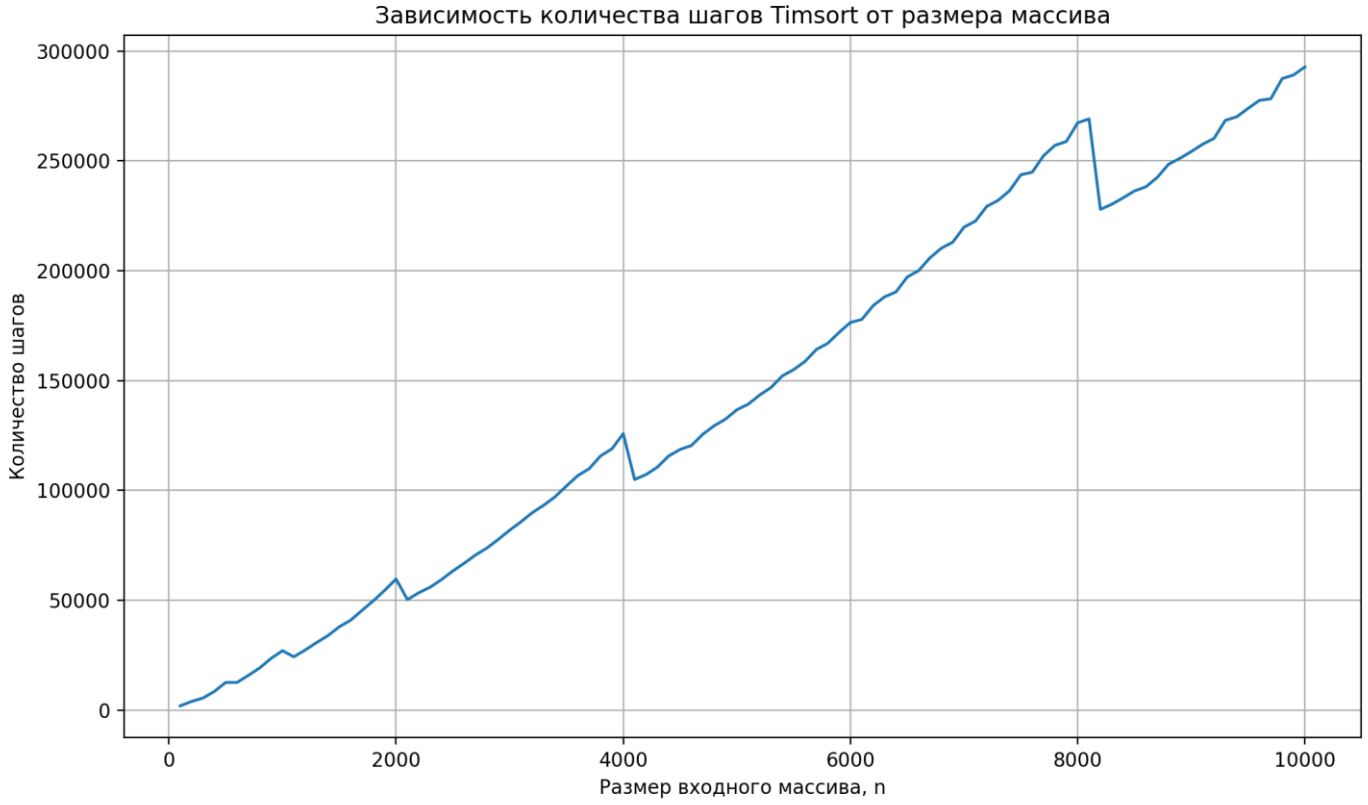
0.181 мс
минимальное время
при n = 100

34.106 мс
максимальное время
при n = 10 000

Что видно по графику:

рост времени плавный;
есть небольшие пики,
но впринцеп устойчиво.

Результат 2: шаги и сложность



Теоретическая сложность

лучший случай — O(n)
средний случай — O(n log n)
худший случай — O(n log n)

1936 … 292 719 шагов

диапазон измерений в эксперименте

Вывод:
практический график близок к O(n log n),
а значит результаты согласуются с теорией.

Итоги работы

- я изучила идею TimSort и реализовала алгоритм на Python;
- сгенерировал 100 входных файлов и измерил работу сортировки по двум критериям;
- получил графики времени и количества шагов;
- по эксперименту видно, что рост близок к O(n log n), а на частично упорядоченных данных TimSort особенно удобен.